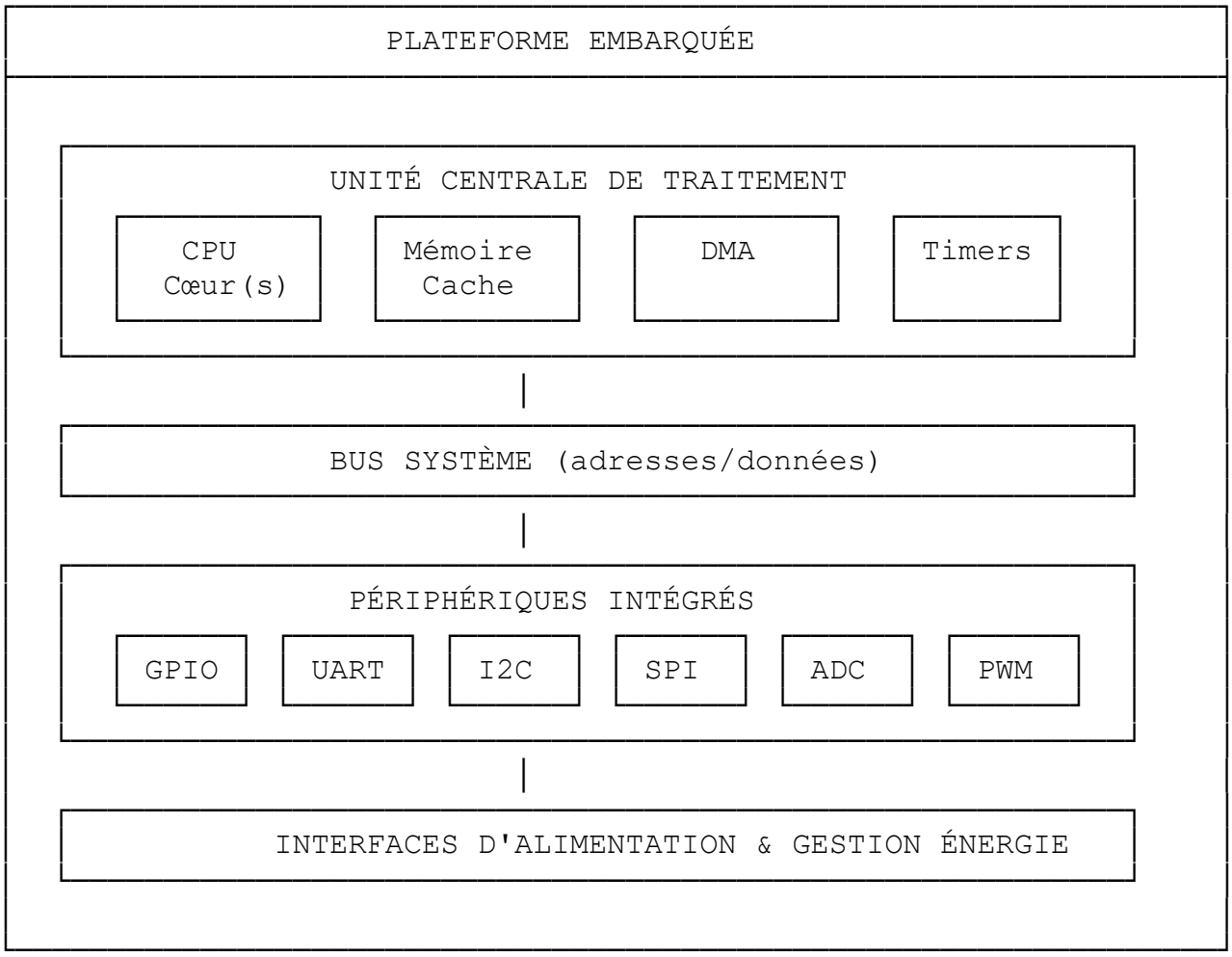
CHAPITRE 2 : PLATEFORMES MATÉRIELLES POUR SYSTÈMES EMBARQUÉS ET IoT

2.1 INTRODUCTION À L'ARCHITECTURE DES PLATEFORMES EMBARQUÉES

2.1.1 Vue d'ensemble architecturale

Une plateforme matérielle pour systèmes embarqués repose sur une architecture organisée autour de trois grands blocs fonctionnels :



2.1.2 Classification des plateformes

Type	Caractéristiques	Exemples
Microcontrôleur (MCU)	CPU, mémoire, périphériques intégrés sur une seule puce	ATmega328P, PIC16F
Microprocesseur (MPU)	CPU seule, mémoire externe nécessaire	PowerPC, ARM9
System-on-Chip (SoC)	Intégration complexe incluant CPU, GPU, périphériques	BCM2711, ESP32-S3

2.2 TECHNOLOGIES DE PROCESSEURS

2.2.1 Architecture des processeurs pour l'embarqué

Architecture Harvard vs Von Neumann

Architecture	Caractéristique	Avantage	Inconvénient
Von Neumann	Bus unique pour instructions et données	Simplicité	Goulot d'étranglement
Harvard	Bus séparés instructions/données	Parallélisme, sécurité	Complexité matérielle

Note : Les processeurs modernes utilisent souvent une architecture **Harvard modifiée** (caches séparés, mémoire unifiée) .

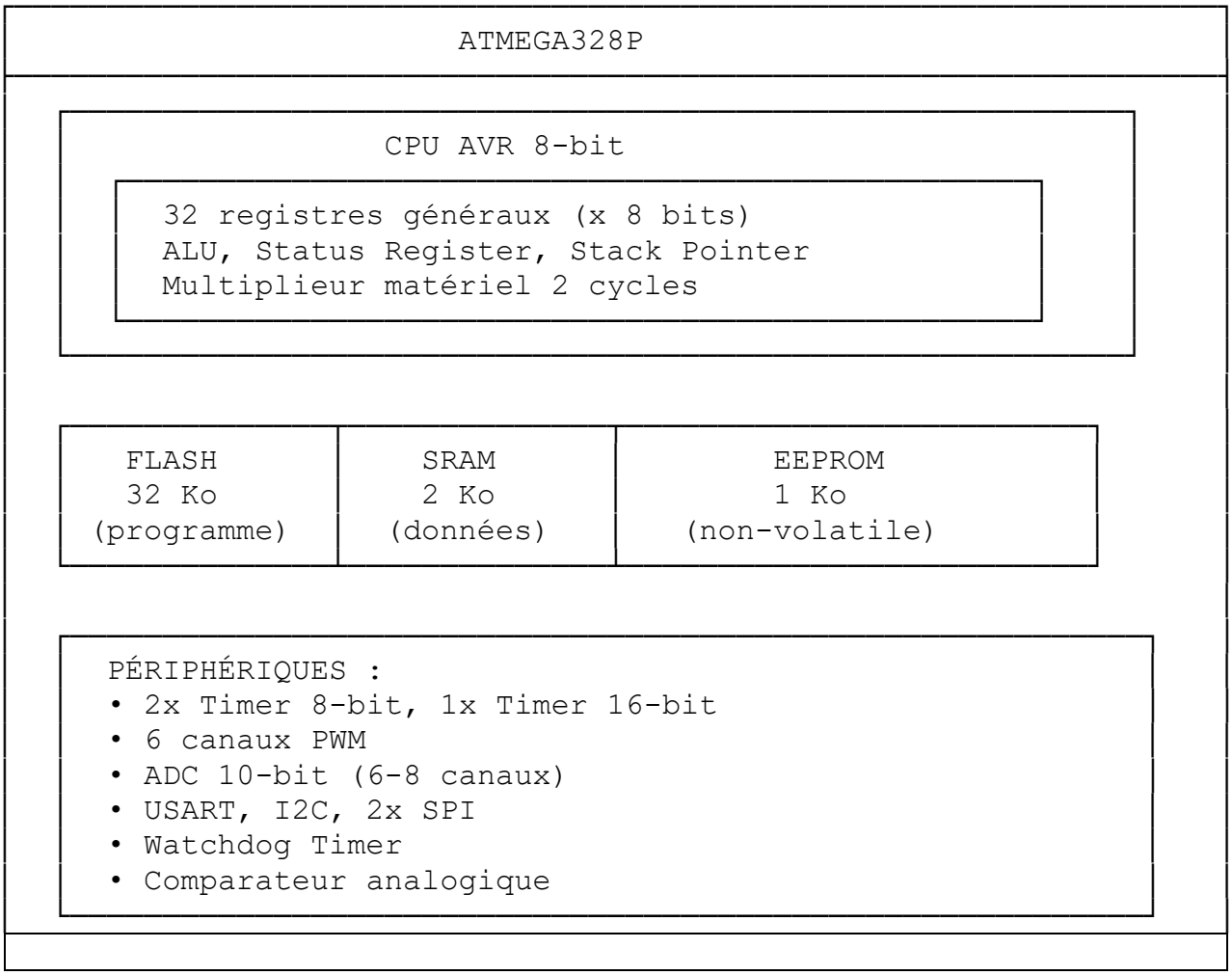
Jeu d'instructions (ISA)

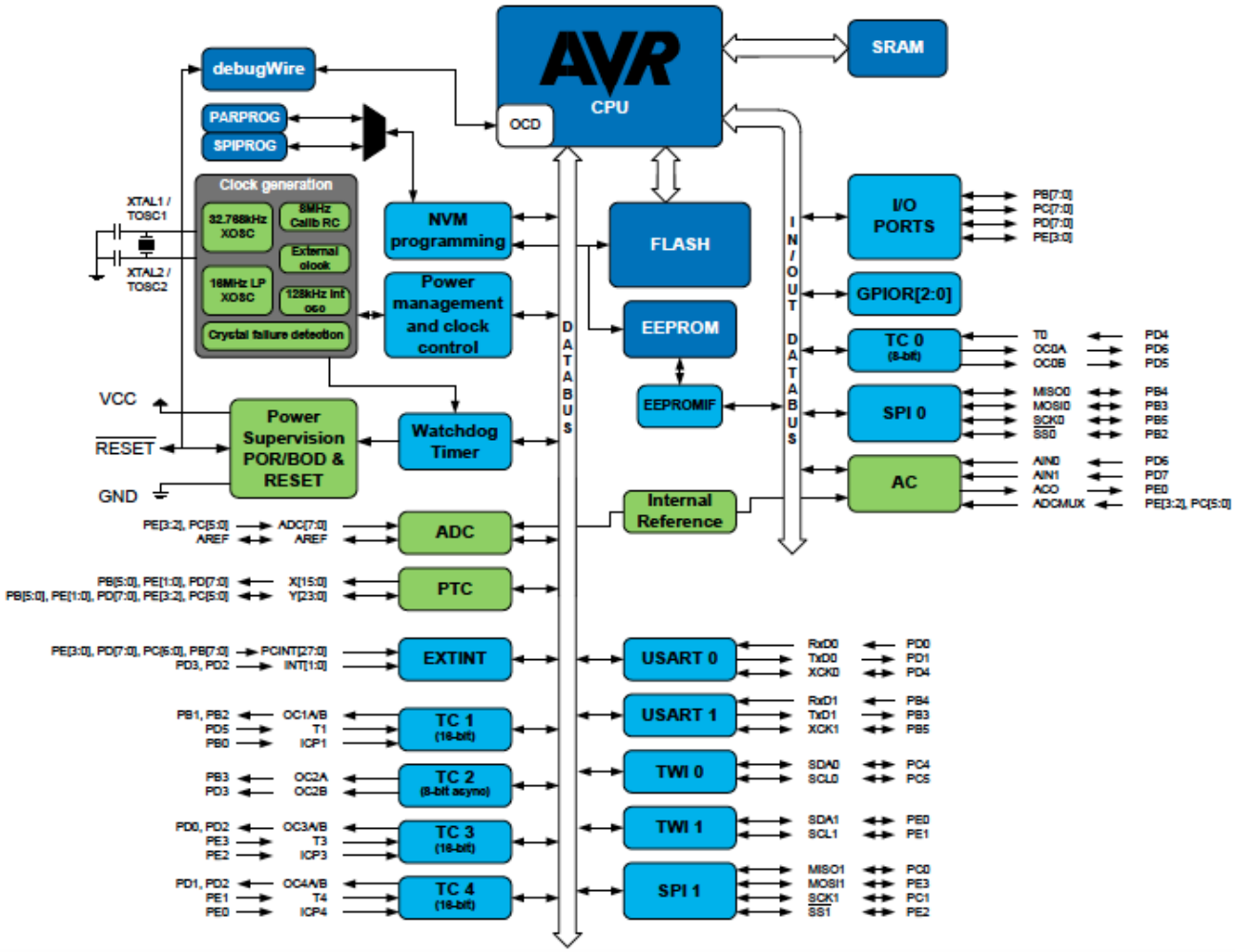
- **RISC (Reduced Instruction Set Computer)** : Instructions simples, exécution rapide (AVR, ARM, RISC-V)
- **CISC (Complex Instruction Set Computer)** : Instructions complexes, code plus compact (x86)

2.2.2 Étude de cas 1 : ATMEGA328P (Microchip)

Architecture générale

L'ATmega328P est un microcontrôleur 8-bit basé sur l'architecture AVR® RISC avancée.





Caractéristiques techniques

- **Fréquence** : Jusqu'à 20 MHz
- **Performance** : 20 MIPS à 20 MHz (1 MIPS par MHz)
- **Tension d'alimentation** : 1,8V - 5,5V
- **Courant en fonctionnement** : 0,2 mA à 1 MHz / 1,8V
- **Courant en veille (Power-Down)** : 0,1 µA
- **Lignes I/O** : 23 programmables
- **Courant max par I/O** : 40 mA

Gestion de l'alimentation

L'ATmega328P propose 6 modes de sommeil :

1. **Idle** : CPU stoppé, périphériques actifs
2. **ADC Noise Reduction** : Réduction bruit pour conversions ADC
3. **Power-Down** : Seulement watchdog et external interrupts

4. **Power-Save** : Comme Power-Down + timer asynchrone actif
5. **Standby** : Oscillateur cristal maintenu
6. **Extended Standby** : Comme Standby + timer actif

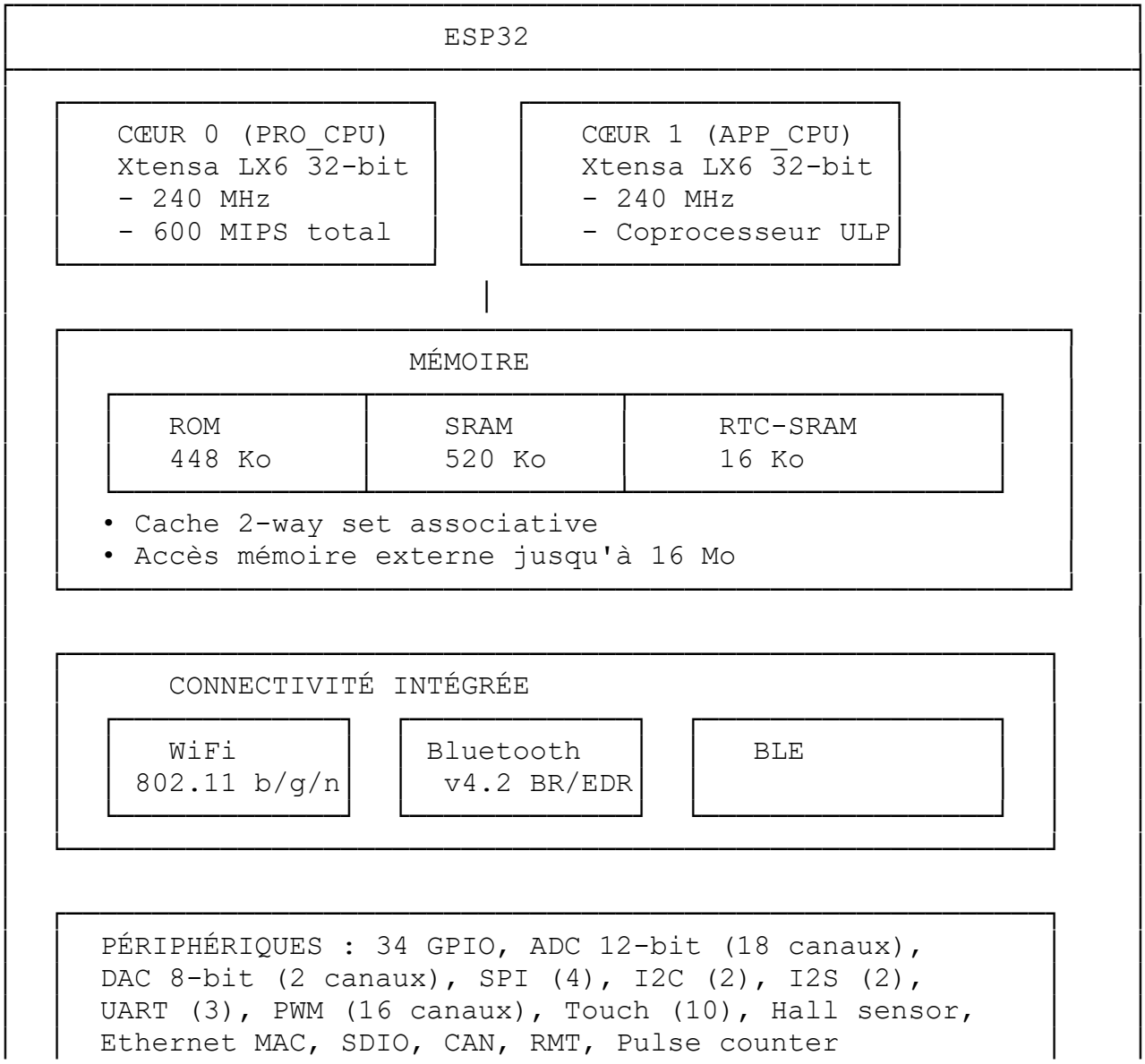
Architecture RISC

- 131 instructions puissantes
- La plupart exécutées en 1 cycle
- Architecture entièrement statique

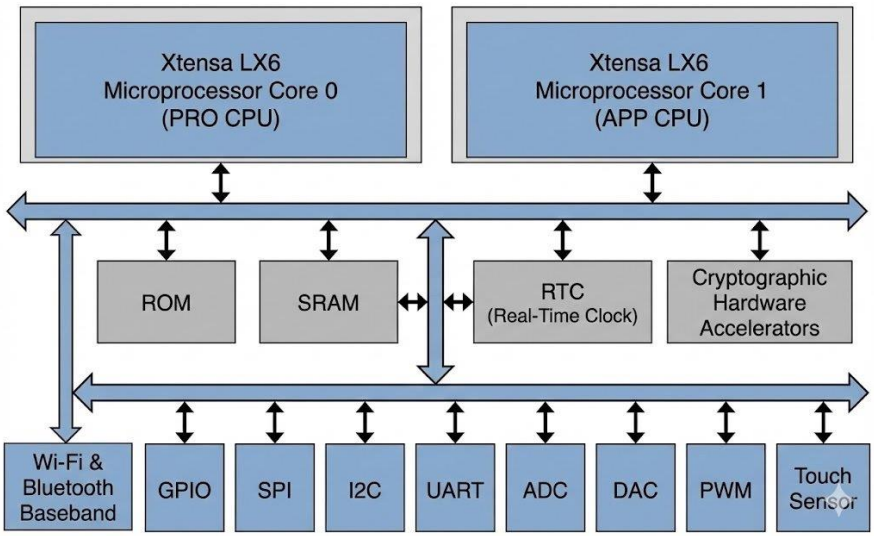
2.2.3 Étude de cas 2 : Xtensa LX6 (ESP32)

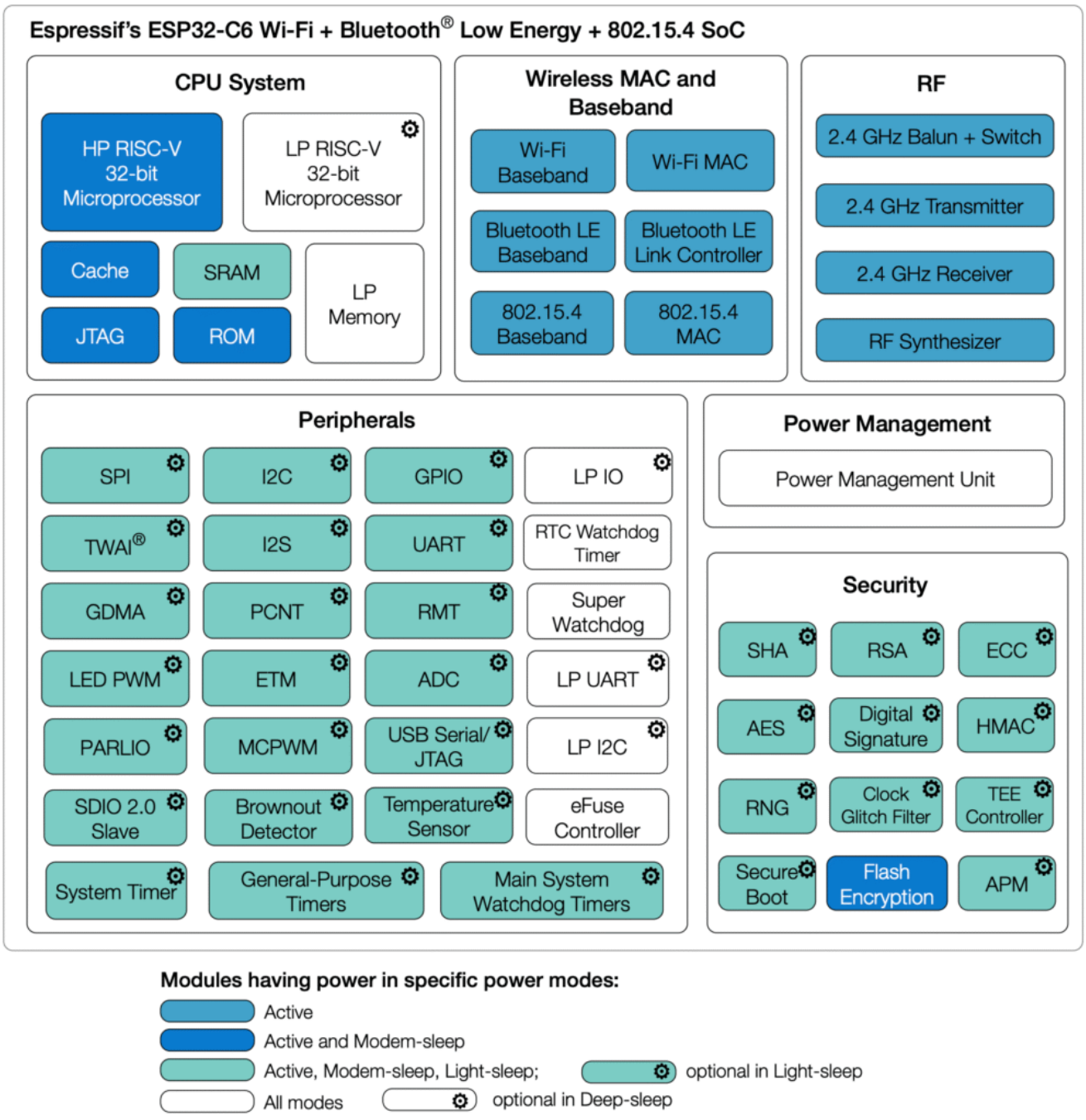
Architecture générale

L'ESP32 intègre un cœur Xtensa LX6 32-bit, disponible en versions dual-core ou single-core.



ESP32 DUAL-CORE ARCHITECTURE DIAGRAM





Architecture mémoire complexe

L'ESP32 utilise une architecture Harvard avec des espaces d'adressage distincts :

- **Bus instruction** : Accès au code
- **Bus données** : Accès aux données
- **Cache** : Pour accès mémoire externe (flash/SPIRAM)
- **RTC memory** : Conservation données en deep sleep

Caractéristiques techniques

- **Fréquence typique** : 240 MHz (160 MHz sur certains modèles)
- **Performance** : Jusqu'à 600 MIPS (dual-core)
- **Tension** : 2,3V - 3,6V (typique 3,3V)
- **Courant** :
    - Actif (WiFi) : ~180 mA
    - Modem sleep : ~20-50 mA
    - Deep sleep : ~10 µA (avec RTC timer)
    - Deep sleep avec ULP : ~150 µA
- **Courant I/O** : 40 mA max par broche

Coprocesseur ULP (Ultra Low Power)

Permet d'exécuter du code en deep sleep pour surveiller des capteurs sans réveiller le CPU principal.

2.2.4 Étude de cas 3 : Broadcom BCM2711 (Raspberry Pi 4)

Architecture générale

Le BCM2711 est un SoC 64-bit conçu pour des applications nécessitant une puissance de calcul élevée.

BCM2711 (Raspberry Pi 4)

CPU ARM Cortex-A72

Cœur 0 1,5 GHz	Cœur 1 1,5 GHz
Cœur 2 1,5 GHz	Cœur 3 1,5 GHz

4 cœurs 64-bit
avec caches L1/L2
partagé

GPU VideoCore VI

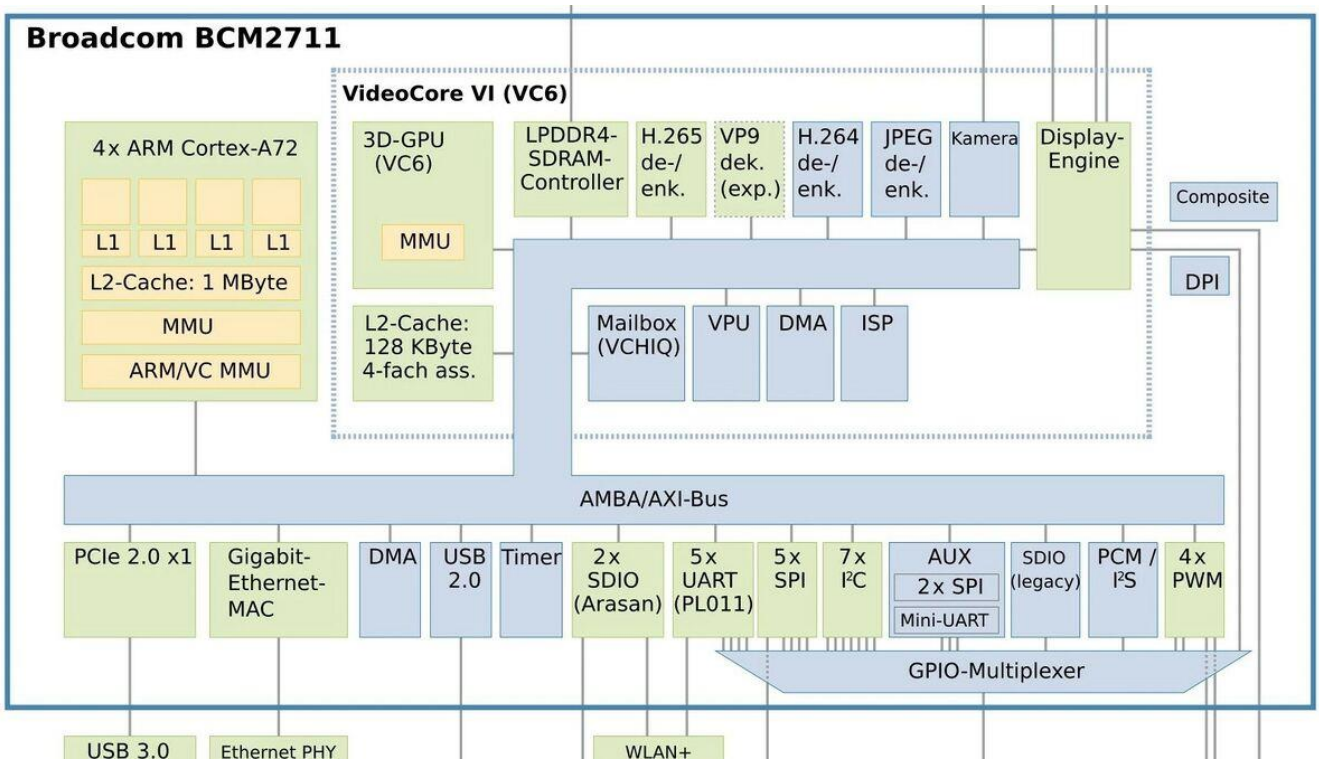
- OpenGL ES 3.0
- H.265 (4Kp60 decode), H.264 (1080p60 decode/encode)
- Support 2 écrans 4K

MÉMOIRE

- Interface LPDDR4-3200 (avec ECC)
- Capacités : 1, 2, 4 ou 8 Go
- Cache L1 : 32 Ko instructions + 32 Ko données par cœur
- Cache L2 : 1 Mo partagé

PÉRIPHÉRIQUES :

- 46 GPIO
- 2x USB 3.0, 2x USB 2.0
- 2x HDMI 2.0
- MIPI DSI (2x) et CSI (2x)
- Ethernet Gigabit
- PCIe Gen 2



Caractéristiques techniques

- **Fréquence CPU** : 1,5 GHz
- **Architecture** : ARMv8-A (64-bit)
- **Tension** : 3,3V (I/O), 1,8V (core)
- **RAM** : Jusqu'à 8 Go LPDDR4-3200
- **Stockage** : Carte microSD ou eMMC optionnel
- **Connectivité intégrée** : WiFi 802.11b/g/n/ac, Bluetooth 5.0
- **Courant I/O** : 16 mA max par GPIO

Particularités

- Documentation technique limitée (peu de détails publics sur les périphériques)
- Conçu pour Linux embarqué, pas pour le bare-metal
- Gestion d'énergie complexe via le firmware VideoCore

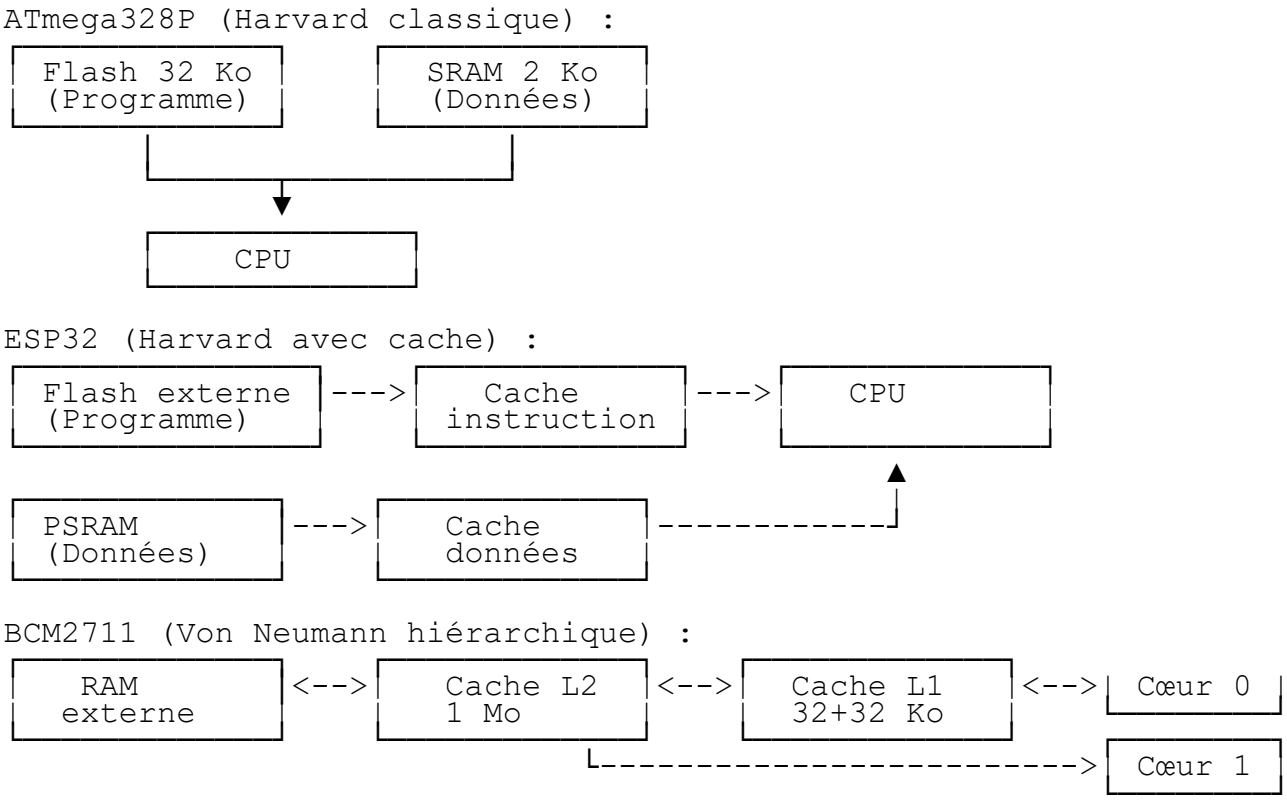
2.3 COMPARAISON DES ARCHITECTURES FONCTIONNELLES

2.3.1 Tableau comparatif détaillé

Caractéristique	ATmega328P	ESP32 (LX6)	BCM2711 (RPi 4)

Architecture	8-bit AVR RISC	32-bit Xtensa LX6	64-bit ARM Cortex-A72
Cœurs	1	2 (ou 1)	4
Fréquence max	20 MHz	240 MHz	1,5 GHz
Performance	20 MIPS	600 MIPS	~13 500 DMIPS
Mémoire interne	2 Ko SRAM + 32 Ko Flash	520 Ko SRAM + ROM	Cache uniquement
Mémoire externe	Jusqu'à 64 Ko	Jusqu'à 16 Mo Flash + 8 Mo PSRAM	Jusqu'à 8 Go LPDDR4
Consommation actif	5-15 mA @5V	80-180 mA @3,3V	600-1200 mA @5V
Consommation veille	0,1 µA (Power-Down)	10 µA (Deep sleep)	~100 mA (suspend)
OS typique	Aucun / RTOS léger	FreeRTOS / ESP-IDF	Linux (Raspberry Pi OS)

2.3.2 Architecture mémoire comparée



2.4 CARACTÉRISTIQUES ÉLECTRIQUES ET DYNAMIQUES

2.4.1 Paramètres électriques fondamentaux

Paramètre	ATmega328P	ESP32	Raspberry Pi 4
VDD (alimentation)	1,8V - 5,5V	2,3V - 3,6V	5V (via USB-C)
Niveau logique I/O	Tension VDD	3,3V	3,3V
VIH min (entrée haut)	0,6 × VDD	0,75 × VDD	2,0V
VIL max (entrée bas)	0,3 × VDD	0,25 × VDD	0,8V
VOH min (sortie haut)	VDD - 0,5V	VDD - 0,4V	2,6V
VOL max (sortie bas)	0,6V	0,4V	0,4V
IOL/IOH max par broche	40 mA	40 mA	16 mA
Courant total GPIO	200 mA	Non spécifié	50 mA (3,3V rail)

2.4.2 Contraintes dynamiques

ATmega328P

- **Fréquence max** : 20 MHz @ 4,5-5,5V ; 10 MHz @ 2,7-5,5V ; 4 MHz @ 1,8-5,5V

- **Temps montée/descente** : Quelques ns (dépend de la charge)
- **Capacité de charge** : Jusqu'à 20 pF par broche

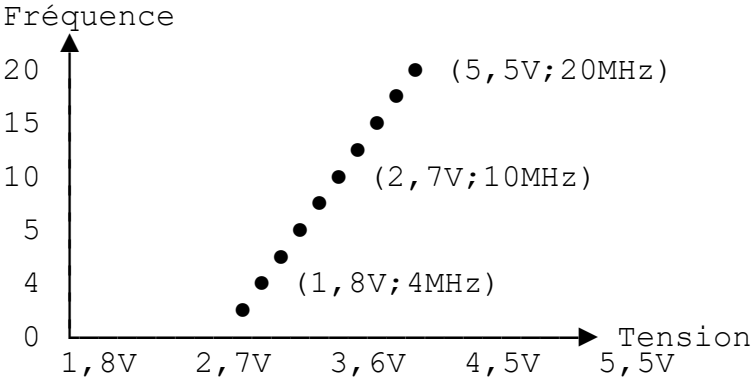
ESP32

- **Fréquence variable** : 80 / 160 / 240 MHz
- **Temps montée/descente** : 2-5 ns (réglable)
- **Driver strength** : Configurable (faible, moyen, fort)

Raspberry Pi 4

- **Fréquence fixe** : GPIO contrôlés par le SoC
- **Temps montée/descente** : ~2 ns
- **Slew rate control** : Possible via software
- **Attention** : La sortie peut chuter sous charge

2.4.3 Courbe tension-fréquence (ATmega328P)



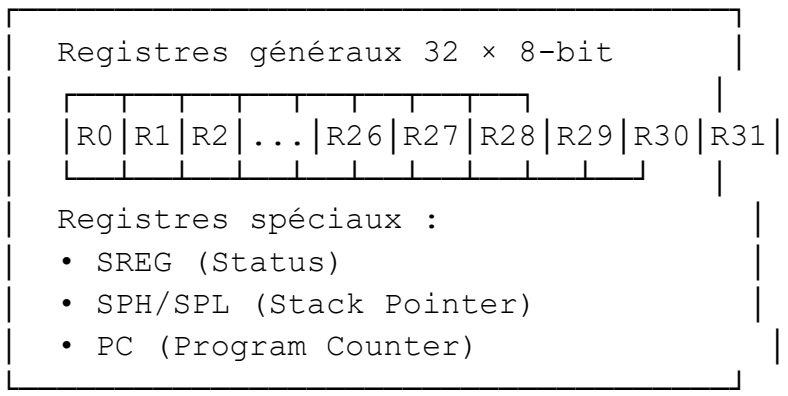
2.5 PRÉSENTATION DES PRINCIPAUX COMPOSANTS DES PROCESSEURS

2.5.1 Unité Arithmétique et Logique (ALU)

Processeur	Taille	Opérations	Particularités
ATmega328P	8-bit	Addition, soustraction, ET, OU, XOR, décalages	Multiplieur 2 cycles
ESP32	32-bit	Opérations entières complètes + MAC	SIMD pour DSP
BCM2711	64-bit	Opérations entières + flottantes (FPU)	NEON SIMD, FPU

2.5.2 Banques de registres

ATmega328P :



ESP32 / BCM2711 :

└── Architecture ARM : 16 registres 32/64-bit

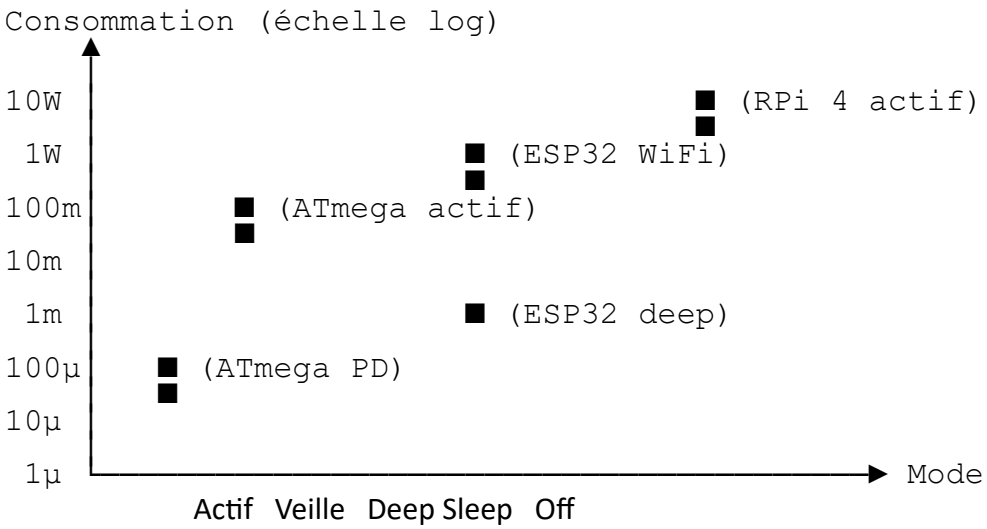
- R0-R12 : Usage général
- R13 (SP) : Stack Pointer
- R14 (LR) : Link Register
- R15 (PC) : Program Counter
- CPSR : Current Program Status Register

2.5.3 Gestionnaires d'interruptions

Processeur	Vecteurs	Niveaux priorité	Interruptions externes
ATmega328P	26 vecteurs	Fixe par vecteur	2 INT + 24 pin change
ESP32	32 niveaux	32 niveaux programmables	Jusqu'à 64 sources
BCM2711	GIC-400	Jusqu'à 256 niveaux	Multiples sources

2.6 GESTION DE L'ALIMENTATION

2.6.1 Modes de consommation comparés



2.6.2 Stratégies de gestion d'énergie

ATmega328P

- **Idle** : CPU stoppé, périphériques actifs (consommation réduite de 80%)
- **Power-Down** : Oscillateurs stoppés, réveil par interruption externe (consommation : 0,1 µA)
- **Power-Save** : Timer asynchrone actif pour RTC
- **BOD (Brown-Out Detection)** : Reset si tension trop basse

ESP32

- **Active** : CPU à pleine vitesse, radios actives
- **Modem sleep** : CPU actif, radio éteinte
- **Light sleep** : CPU stoppé, réveil par timer ou GPIO
- **Deep sleep** : Seulement RTC et ULP (10 µA)
- **ULP coprocessor** : Exécution de code en deep sleep

Raspberry Pi 4

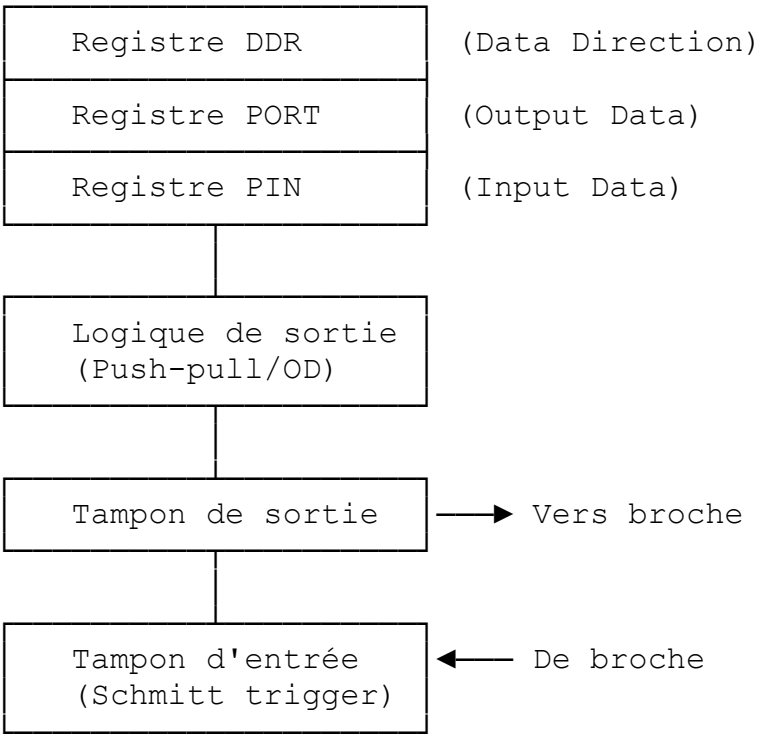
- **Active** : Tous cœurs actifs
- **Governor** : Ajustement dynamique fréquence/tension
- **Suspend** : RAM maintenue, CPU stoppé
- **Halt** : État basse consommation (non documenté officiellement)

2.6.3 Techniques d'optimisation énergétique

Technique	ATmega328P	ESP32	RPi 4
Horloge variable	Limitée (prescaler)	Oui (80-240 MHz)	Oui (CPUFreq)
Mise en veille automatique	Manuelle	Manuelle + ULP	Manuelle
Power gating	Non	Oui (RTC domain)	Oui (cores)
Clock gating	Périphériques	Périphériques	Avancé

2.7 GESTION DES PORTS D'ENTRÉES/SORTIES

2.7.1 Structure d'un port GPIO typique



2.7.2 Modes de fonctionnement des GPIO

Mode	Description	ATmega328P	ESP32	RPi 4
Input floating	Entrée haute impédance	Oui	Oui	Oui
Input pull-up	Entrée avec résistance pull-up	Oui (20-50 kΩ)	Oui (configurable)	Oui (50 kΩ)
Input pull-down	Entrée avec résistance pull-down	Non	Oui	Oui (50 kΩ)
Output push-pull	Sortie 0/VDD	Oui	Oui	Oui
Output open-drain	Sortie 0 ou haute impédance	Logiciel	Oui	Oui
Alternate function	Connecté à périphérique	Oui	Oui	Oui

2.7.3 Configuration des GPIO (exemples)

ATmega328P

```
// Configuration d'une broche en sortie
DDRB |= (1 << PB0);    // PB0 en sortie
PORTB |= (1 << PB0);   // Mettre à 1

// Configuration en entrée avec pull-up
DDRB &= ~(1 << PB1);   // PB1 en entrée
PORTB |= (1 << PB1);   // Activer pull-up

// Lecture d'entrée
if (PINB & (1 << PB1)) {
   // Broche à 1
}
```

ESP32

c

```
// Avec Arduino
pinMode(2, OUTPUT);
digitalWrite(2, HIGH);
pinMode(4, INPUT_PULLUP);
int val = digitalRead(4);

// Avec ESP-IDF
gpio_set_direction(GPIO_NUM_2, GPIO_MODE_OUTPUT);
gpio_set_level(GPIO_NUM_2, 1);
gpio_set_pull_mode(GPIO_NUM_4, GPIO_PULLUP_ONLY);
```

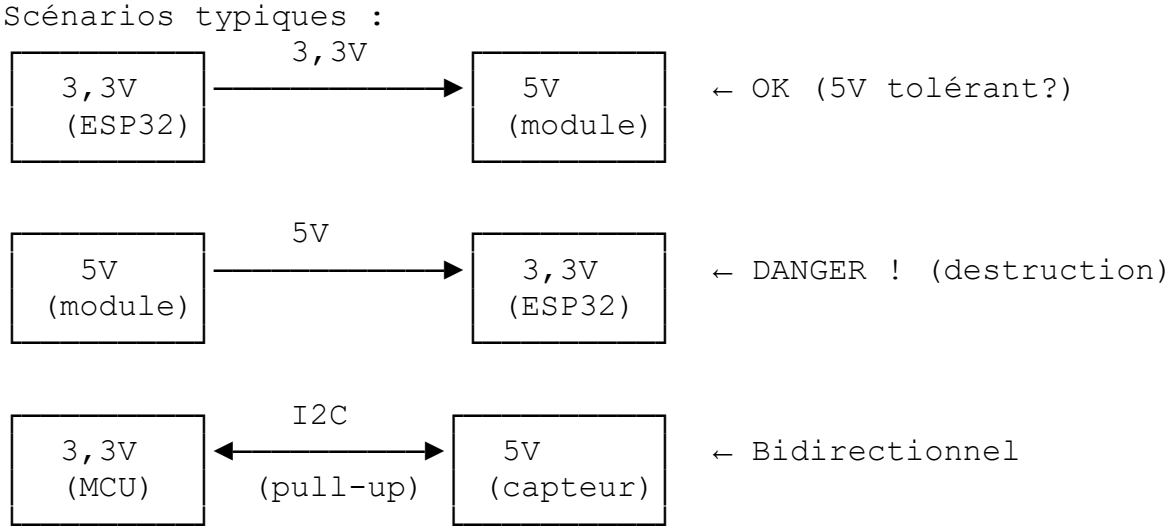
Raspberry Pi (Python)

```
python
import RPi.GPIO as GPIO
GPIO.setmode(GPIO.BCM)
GPIO.setup(18, GPIO.OUT)  # Sortie
GPIO.output(18, GPIO.HIGH)
GPIO.setup(23, GPIO.IN, pull_up_down=GPIO.PUD_UP)  # Entrée pull-up
GPIO.input(23)
```

2.8 ADAPTATION DE NIVEAU DE TENSION ET DE PUISSANCE

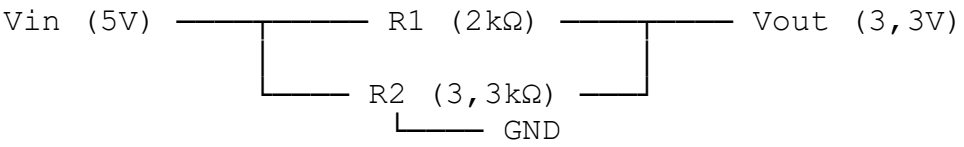
2.8.1 Problématique des niveaux de tension

Les systèmes embarqués doivent souvent interfacer des composants fonctionnant à des tensions différentes.



2.8.2 Techniques d'adaptation de niveau

Pont diviseur résistif (unidirectionnel)

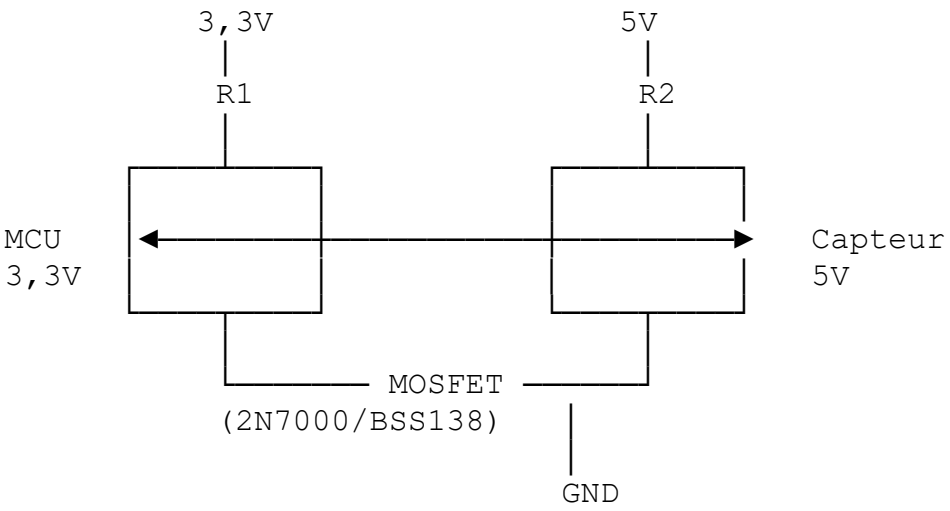


Formule : Vout = Vin × R2/(R1+R2)

Exemple : 5V × 3,3k/(2k+3,3k) = 3,3V

Avantages : Simple, pas cher
Inconvénients : Unidirectionnel, lent, consommation

Convertisseur de niveau à MOSFET



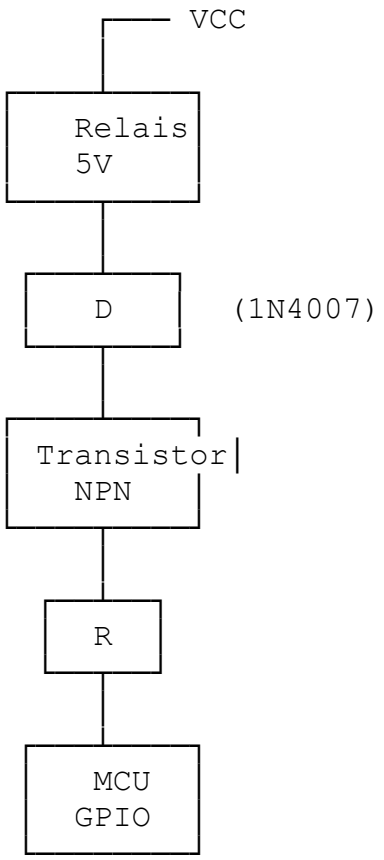
Circuits intégrés spécialisés

Circuit	Type	Canaux	Direction	Fréquence max
TXB0108	Auto-sensing	8	Bidirectionnel	100 MHz
TXS0108	Push-pull	8	Bidirectionnel	40 MHz
PCA9306	I2C spécifique	2	Bidirectionnel	400 kHz
ADG3308	Contrôle DIR	8	Configurable	50 MHz

2.8.3 Adaptation de puissance

Commande de charges inductives (relais, moteurs)

Protection par diode de roue libre :



Circuits intégrés de puissance

Circuit	Type	Tension	Courant	Protection
ULN2003	Darlington array	50V	500 mA/canal	Diodes intégrées
L293D	Pont en H	36V	600 mA	Thermique
MOSFETs	Discrets	Variable	Variable	Externe
Relais statiques	Isolation	Réseau	Jusqu'à plusieurs A	Opto-isolation

---

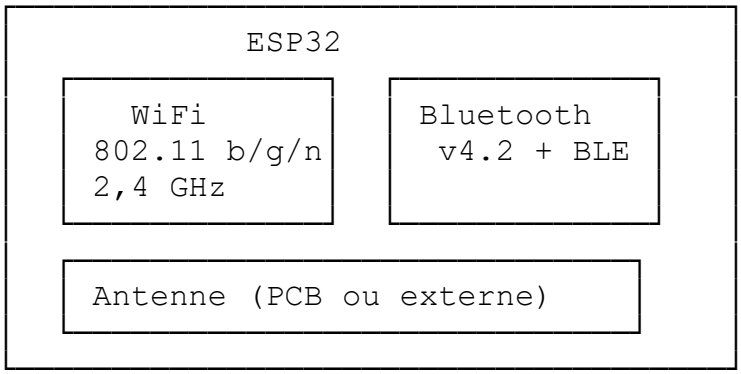
2.9 GESTION DE LA CONNECTIVITÉ ET DES COMMUNICATIONS

2.9.1 Interfaces filaires comparées

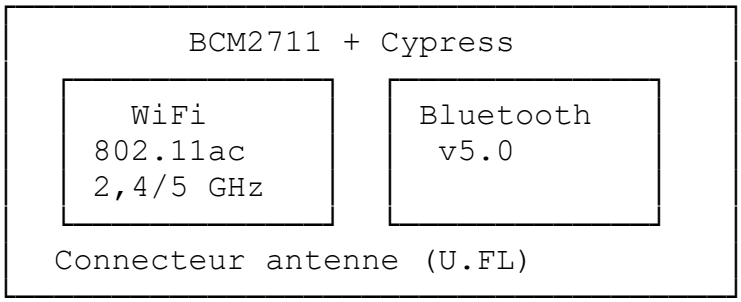
Interface	ATmega328P	ESP32	BCM2711	Vitesse max
UART	1 (USART)	3	4+	Jusqu'à 5 Mbps
I2C	1 (TWI)	2	Plusieurs	400 kHz / 1 MHz
SPI	2	4	Plusieurs	10 MHz / 80 MHz
CAN	Non	1 (TWAI)	Non	1 Mbps
USB	Non	Non (sauf S2/S3)	4 ports	480 Mbps / 5 Gbps
Ethernet	Non	MAC (PHY externe)	1 Gbps	100 Mbps / 1 Gbps

2.9.2 Connectivité sans fil

ESP32 (intégré) :



Raspberry Pi 4 (intégré) :



ATmega328P : Nécessite module externe (ESP8266, nRF24, HC-05)

2.9.3 Protocoles supportés

Protocole	ATmega328P	ESP32	RPi 4
TCP/IP	Externe (module)	Intégré (lwIP)	Intégré (Linux)
MQTT	Librairie	Librairie	Librairie
HTTP	Librairie	Librairie	Serveur/client
WebSocket	Limitée	Oui	Oui
Bluetooth SPP	Non	Oui	Oui
BLE	Non	Oui	Oui

2.10 APPLICATIONS TYPES POUR CHAQUE PLATEFORME

2.10.1 ATmega328P (Arduino Uno)

Caractéristiques :

- Simple, robuste, tolérant 5V
- Faible consommation
- Programmation simple

Applications typiques:

- **Prototypage rapide** : Projets éducatifs, premiers pas
- **Capteurs simples** : Lecture température, humidité, distance
- **Actionneurs basiques** : LED, relais, petits moteurs
- **Automation domestique simple** : Arrosage automatique, éclairage
- **Projets scolaires** : Robots suiveurs de ligne, stations météo

Exemple : Station météo avec affichage LCD

cpp

```
#include <LiquidCrystal.h>

LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

float temperature = 22.5;


void setup() {
  lcd.begin(16, 2);
  lcd.print("Temperature:");
}


void loop() {
  lcd.setCursor(0, 1);
  lcd.print(temperature);
  lcd.print(" C");
  delay(1000);
}
```

2.10.2 ESP32

Caractéristiques :

- Connectivité intégrée WiFi/Bluetooth
- Dual-core puissant
- Faible coût

Applications typiques:

- **IoT connecté** : Capteurs communicants, objets connectés
- **Serveur web embarqué** : Interface de contrôle local
- **Gateway IoT** : Agrégation de données de capteurs
- **Audio streaming** : Récepteurs Bluetooth, speakers connectés
- **Surveillance à distance** : Caméras IP (avec module), capteurs

Exemple : Serveur web simple

cpp

```
#include <WiFi.h>
#include <WebServer.h>

WebServer server(80);

void handleRoot() {
  server.send(200, "text/html",
    "<h1>ESP32 Web Server</h1>"
    "<p>Température: 22.5°C</p>");
}

void setup() {
  WiFi.begin("SSID", "password");
  server.on("/", handleRoot);
  server.begin();
}

void loop() {
  server.handleClient();
}
```

2.10.3 Raspberry Pi 4

Caractéristiques :

- Puissance de calcul élevée
- Linux complet
- Connectivité riche

Applications typiques:

- **Serveur embarqué** : Serveur web local, base de données
- **Station multimédia** : Kodi, RetroPie
- **Vision par ordinateur** : Traitement d'images, reconnaissance
- **Gateway IoT avancé** : Agrégation, edge computing
- **Contrôleur industriel** : IHM, supervision

Exemple : Serveur Flask simple

python

```
from flask import Flask
import RPi.GPIO as GPIO

app = Flask(__name__)

@app.route('/')
def home():
    return "<h1>RPi Control</h1>"

@app.route('/led/on')
def led_on():
    GPIO.output(18, GPIO.HIGH)
    return "LED ON"

if __name__ == '__main__':
```

```
app.run(host='0.0.0.0', port=80)
```

2.11 CRITÈRES DE CHOIX DES PLATEFORMES MATÉRIELLES

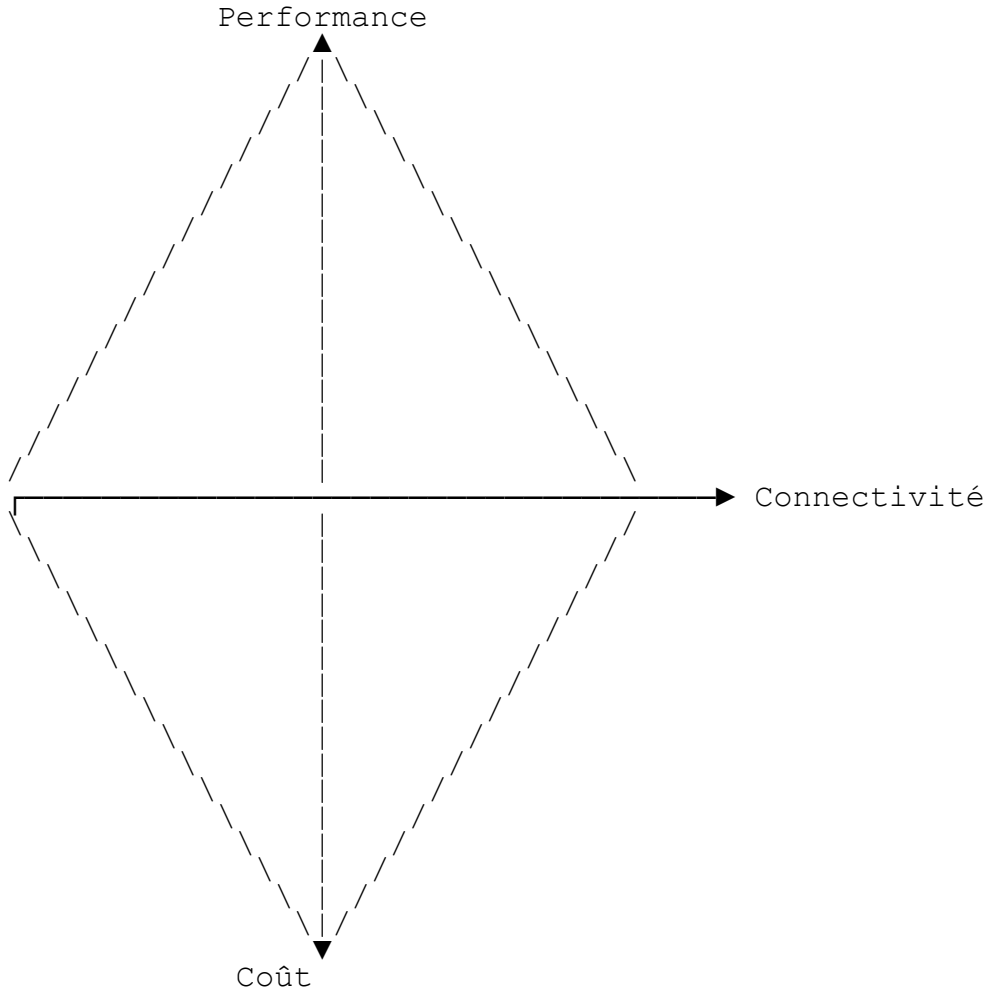
2.11.1 Arbre de décision

```
DÉBUT
  │
  ├── Besoin de connectivité intégrée ?
  │    ├── OUI → ESP32 (WiFi/BT) ou RPi (plus puissant)
  │    └── NON → Passer à critère suivant
  │
  ├── Puissance de calcul nécessaire ?
  │    ├── Faible (lecture capteurs) → ATmega328P
  │    ├── Moyenne (traitement IoT) → ESP32
  │    └── Élevée (Linux, vision) → RPi 4
  │
  ├── Contrainte de consommation ?
  │    ├── Critique (batterie années) → ATmega328P
  │    ├── Importante (batterie jours) → ESP32
  │    └── Faible (secteur) → RPi 4
  │
  ├── Environnement d'exécution ?
  │    ├── Industriel (température, vibrations) → ATmega328P/ESP32
  │    └── Bureau/domestique → Tous
  │
  ├── Complexité logicielle ?
  │    ├── Simple (boucle principale) → ATmega328P
  │    ├── RTOS, multitâche → ESP32 (FreeRTOS)
  │    └── Linux, applications complexes → RPi 4
  │
  └── Budget ?
       ├── Très serré (< 5€) → ATmega328P (nu)
       ├── Modéré (10-20€) → ESP32
       └── Plus élevé (50-100€) → RPi 4
```

2.11.2 Tableau de sélection par application

Application	ATmega328P	ESP32	RPi 4	Recommandation
Capteur simple (température, humidité)	★★★★★	★★★	★★	**ATmega328P**
Capteur connecté WiFi/BT	★ (module)	★★★★★	★★★★	**ESP32**
Station météo avec logs	★★★	★★★★	★★★★★	**RPi ou ESP32**
Contrôle moteur DC/Pas à pas	★★★★	★★★★	★★	**ATmega/ESP32**
Interface Homme-Machine tactile	★ (LCD)	★★★	★★★★★	**RPi 4**
Gateway IoT (collecte+upload)	Non	★★★★	★★★★★	**RPi 4**
Objet connecté sur batterie	★★★★★	★★★★	★	**ATmega328P**
Traitement vidéo/reconnaissance	Non	★ (module)	★★★★★	**RPi 4**
Éducation/Apprentissage	★★★★★	★★★★	★★★★★	**ATmega (début) puis RPi**
Produit commercial volume	★★★★★	★★★★★	★★	**ATmega/ESP32**

2.11.3 Compromis fondamentaux



2.12 EXERCICES D'APPLICATION

Exercice 1 : Analyse de spécifications

À partir des caractéristiques techniques fournies dans ce chapitre, comparez les trois plateformes pour les métriques suivantes :

1. Rapport performance/consommation (MIPS/mA)
2. Densité mémoire (Ko de RAM par mm²)
3. Coût par fonction (connectivité, calcul, I/O)

Correction indicative :

ATmega328P : 20 MIPS / 15 mA = 1,33 MIPS/mA

ESP32 : 600 MIPS / 180 mA = 3,33 MIPS/mA

RPi 4 : 13500 MIPS / 800 mA = 16,9 MIPS/mA

Exercice 2 : Choix de plateforme pour un projet

Pour chacun des projets suivants, justifiez le choix de la plateforme la plus adaptée :

Projet A : Thermostat intelligent fonctionnant sur piles (2×AA) pendant 1 an, avec affichage LCD, communication WiFi occasionnelle, contrôle de relais.

Projet B : Système de reconnaissance faciale pour contrôle d'accès, avec caméra HD, base de données locale, interface tactile.

Projet C : Réseau de 50 capteurs de température sans fil, avec transmission toutes les 10 minutes, autonomie 5 ans.

Projet D : Bras robotique contrôlé par joystick, avec 6 servomoteurs, retour de force, interface utilisateur simple.

Exercice 3 : Adaptation de niveau

Concevez un circuit d'interface entre un ESP32 (3,3V) et les périphériques suivants :

1. Capteur ultrason HC-SR04 (5V)
2. Module afficheur LCD I2C (5V)
3. Relais 12V commandé par transistor

Solution partielle :

- Pour HC-SR04 : Pont diviseur sur l'entrée Echo (5V→3,3V)
- Pour I2C : Convertisseur PCA9306 ou MOSFET BSS138

- Pour relais : Optocoupleur + transistor + diode de roue libre

Exercice 4 : Calcul de consommation

Un système basé sur ESP32 fonctionne selon le cycle suivant :

- 100 ms actif (WiFi transmission) à 180 mA
- 10 s en deep sleep à 10 µA

Calculez :

1. La consommation moyenne
2. L'autonomie avec une batterie 2000 mAh

Correction :

Cycle total = 10,1 s

Énergie par cycle = (0,1 × 180) + (10 × 0,01) = 18 + 0,1 = 18,1 mA·s

Courant moyen = 18,1 / 10,1 = 1,79 mA

Autonomie = 2000 / 1,79 = 1117 heures ≈ 46 jours

Exercice 5 : Configuration GPIO

Écrivez le code pour les configurations suivantes :

1. **ATmega328P** : Configurer PB0 en sortie, PB1 en entrée avec pull-up, lire PB1 et recopier sur PB0
2. **ESP32** : Configurer GPIO2 en sortie PWM pour contrôler une LED, GPIO4 en entrée interruption sur front descendant
3. **Raspberry Pi** : Script Python qui alterne une LED sur GPIO18 toutes les secondes

---

2.13 SYNTHÈSE DU CHAPITRE

Points clés à retenir

1. **Trois grandes familles** de plateformes : microcontrôleurs 8-bit (ATmega), SoC connectés 32-bit (ESP32), processeurs application 64-bit (RPi)
2. **Architecture** : Harvard (ATmega, ESP32) vs Von Neumann hiérarchique (RPi)
3. **Compromis fondamental** : Performance ↔ Consommation ↔ Coût
4. **Caractéristiques électriques critiques** : niveaux logiques (5V vs 3,3V), courant disponible par I/O
5. **Adaptation de niveau** : Essentielle pour mélanger composants de tensions différentes

6. **Gestion d'énergie** : Modes veille essentiels pour applications sur batterie
7. **Choix de plateforme** : Dépend de l'application, pas de solution universelle

Tableau récapitulatif

Critère	ATmega328P	ESP32	RPi 4
Facilité débutant	★★★★★	★★★★	★★★
Puissance calcul	★	★★★	★★★★★
Connectivité intégrée	★ (externe)	★★★★★	★★★★
Faible consommation	★★★★★	★★★★	★
Coût	★★★★ (5€)	★★★★★ (3€)	★★ (50€)
Périphériques intégrés	★★	★★★★★	★★★
Support logiciel	★★★★★	★★★★	★★★★★
Industriel	★★★★	★★★	★★

Références principales :

- Microchip, "ATmega328/P Datasheet"
- Espressif, "ESP32 Memory Map"
- Raspberry Pi, "Compute Module 4S Specifications"
- Fab Academy, "Embedded Programming Comparison"

-
-
- Datasheet
- ATMEGA328-PU - MICROCHIP TECHNOLOGY
- The picoPower® ATmega328/P is a low-power CMOS 8-bit microcontroller based on the AVR® enhanced RISC architecture. By executing powerful instructions in a single clock cycle, the ATmega328/P achieves
-
-
- Espressif
- 2024/08/19
- ESP32's family Memory Map 101
- The ESP32 is the Xtensa LX6 dual-core microcontroller, and it is the oldest SoC from the ESP32’s series of SoCs. The SoC has access to external flash memory using the SPI interface.
-
-
- Raspberry Pi
- Buy a Compute Module 4S – Raspberry Pi
- Specifications Broadcom BCM2711 quad-core 64-bit Cortex-A72 (Arm v8) at 1.5GHz 1GB, 2GB, 4GB and 8GB LPDDR4-3200 SDRAM with ECC Options for 8GB, 16GB, or 32GB eMMC flash
-

- fabcloud.io
- Embedded programming
- Parametres | Atmega328p | ESP 32 | Rasberry pi 3 ---|---|---|---|--- 1. | Operating Voltage | 5V | 3.3V | 3.3V 2. | DC current on I/O pins | 40mA | 40mA | 16mA 3. | Digital I/O pins | 14 | 39 | 26
-
-
- Fab Academy
- Embedded Programming:
- Atmega328p ATmega328/P is a low-power CMOS 8-bit microcontroller based on the AVR® enhanced RISC architecture. By executing powerful instructions in a single clock cycle, the ATmega328/P
-
-
- Microchip Technology
- ATmega328P
- Program Memory Size (KB):32 - RAM:2048 - Data EEPROM (bytes):1024 - Pin Count:32 - Operation Voltage Max.(V) ... The high-performance Microchip picoPower® 8-bit AVR® RISC-based microcontroller combines 32 KB ISP Flash memory with read-while-write capabilities, 1024B EEPROM...
-
-
- Espressif
- Chip Series Comparison¶
- Xtensa® dual-core 32-bit LX6 with 600 MIPS (in total); 200 MIPS for ESP32-U4WDH/ESP32-S0WD (single-core variants); 400 MIPS for ESP32-D2WD
-
-
- Raspberry Pi Forums
- A more detailed datasheet for BCM2711 peripherals
- I am very impatient to find out if the BCM2711's PCM/I2S peripheral will support TDM mode (multichannel audio) unlike the BCM2835/7. ... I would like understand the capabilities of BCM2711 PCM/I2S interface.
-
-
- Kiwix
- 2019/03/16
- GPIO only reaches 2,4v when high
- To flash the SleepyPi (ATMEGA328P) I have to pulldown the GPIO 22 from 3,5V to trigger the reset line for the bootloader to accept my code on the UART bus of the Raspberry Pi.
-
-
- 电子工程世界（EEWorld）
- 2024/05/08
- What to choose for entry-level microcontrollers [Copy link]

- As an electronic engineer getting started with microcontrollers, you can choose a microcontroller tha
-